KU LEUVEN

FACULTEIT WETENSCHAPPEN

An Operator for Dynamic Cross-Layer Enforcement of Security Policies in a Cloud-Based Kubernetes Cluster

Quinten LAUWAERT

Promotor: Prof. dr. ir. W. Joosen
DistriNet

Co-promotor: Dr. E. Truyen
DistriNet

Mentor: Dr. E. Truyen
DistriNet

Assessor: Prof. dr. T. Van Cutsem
DistriNet

Thesis submitted to
obtaining the degree of
Master of Science in
Applied Computer Science

Academiejaar 2024-2025

Preface

As I sit here and write these last few words, I can hardly believe I have reached the end of this long journey. I am filled with gratitude, to have been able to make it this far. It wasn't always a certainty for me that I would reach the end of my academic journey successfully.

The last year has been difficult, with a lot of ups and downs and a lot of times where I felt like I hit a brick wall. This was very frustrating for me and for my mentor. Therefore, I want to extend my gratitude to my mentor Eddy Truyen for standing by me when I could not see the path ahead. I want to thank him for always staying motivated to guide me into the right direction and encouraging me to keep going.

I also want to thank my family for consistently being there for me in this challenging time and whose love and encouragement lifted me back up after every stumble. Their continued support and presence has been a light in this journey and was invaluable to me for the completion of this work.

Lastly, I want to thank every single person in my life that has positively influenced me along the way, giving the energy and motivation to keep going.

Quinten

Abstract

The Cloud Native paradigm is quickly gaining a lot of popularity for deploying applications in a fast, scalable and cost-effective way. This can be attributed to the fact that it leverages the fast spin-up times of containers and the multi-tenant nature of cloud platforms, enabling multiple customers to utilize the same, shared physical resources. However, there is a security tradeoff to this efficient and cost-effective deployment of applications, due to known container-image vulnerabilities and less isolated nature of containers, allowing for potential container escapes into the host. In the case where an attacker exploits such vulnerabilities and is able to escape from the container context, there could exist possible *lateral movement* attacks, where the attacker attempts to gain access to the other nodes in the platform.

In this post-exploitation scenario, it is exactly this movement that we intend to reduce with a tool called Grasshopper, designed in previous work. Grasshopper works by dynamically configuring the cloud platform's security rules, in response to pod scheduling decisions and network policies that are in place. It enforces the principle of least-privilege on the cloud platform's security rules, hereby limiting the amount of *lateral movement* attacks possible on the nodes in the platform.

In this thesis we review its design and its inner workings into limiting such attacks. We implement a baseline version (V1) of the program as well as a second version (V2) where we apply a pattern, called the Operator pattern to the existing version. We do this by implementing it, using the Kopf framework, where we leverage its concurrency model in order to improve the scalability of the tool.

Additionally, we implement a third version (V3) that uses persistence of GH's most vital information, in an attempt to improve the tool's reliability. This feature should improve the recovery time of the program, in case of failure, hereby improving its reliability.

Then, we perform an evaluation of the three versions on their scalability and reliability features, in an attempt to discover which aspects of applying the Operator pattern are worthwhile to include in its design. Moreover, we show the effectiveness of the tool in reducing the aforementioned *lateral movement* attacks, by performing an experiment simulating such attacks in a cluster environment where an instance of GH running vs. an environment where no instance of GH is running.

Samenvatting

Het Cloud Native paradigma wint snel aan populariteit voor het hosten van moderne applicaties in een snelle, schaalbare en kostenefficiënte manier. Dit komt omdat het de snelle spin-up tijden van containers benut en dat het toelaat dat meerdere gebruikers tegelijkertijd dezelfde fysieke middelen gebruiken. Toch is er een security tradeoff gekoppeld aan deze snelle en efficiënte manier van applicaties te hosten, door het feit dat er gekende container-image kwetsbaarheden bestaan en het feit dat deze containers minder afgeschermd zijn dan virtuele machines, wat toelaat voor container ontsnappingen naar de host. In het geval waarin een aanvaller deze kwetsbaarheden misbruikt en de container context kan ontsnappen, kan er sprake zijn van mogelijke *lateral movement* aanvallen, waarin de aanvaller probeert access te krijgen tot de andere nodes in het platform.

In dit post-exploitatie scenario, is het exact deze laterale beweging dat we proberen te reduceren met een tool genaamd Grasshopper, wat al ontwikkeld is in vorig werk. Grasshopper werkt door dynamisch de security regels van het cloud platform te configureren, door te kijken naar de network policies die bestaan en pods die worden gescheduled op nodes. Het dwingt het principe van *least-privilege* af op de security regels van het cloud platform, waarbij het, het aantal mogelijke *lateral movement* aanvallen op de nodes in het platform, reduceert.

In deze thesis bekijken we het design van GH en haar innerlijke werking, zodat het zulke aanvallen kan limiteren. We implementeren een baseline versie (V1) van het programma en ook een tweede versie (V2) waarbij we een patroon, genaamd het Operator patroon, toepassen op de huidige versie. We doen dit door het te implementeren met het Kopf framework, waarbij we haar concurrency model benutten om de schaalbaarheid van de tool te verbeteren.

Bijkomend, implementeren we nog een derde versie (V3) dat gebruik maakt van het wegschrijven van GH's meest vitale informatie naar een permanente gegevensopslag, in een poging om de tool's betrouwbaarheid te verbeteren.

Dan voeren we een evaluatie uit van de drie gemaakte versies op hun schaalbaarheids- en betrouwbaarheids-kenmerken, in een poging om te ontdekken welke aspecten van het toepassen van het Operator patroon de moeite waard zijn om te betrekken in het design van GH. Bovendien, tonen we de effectiviteit van de tool aan, in het reduceren van de voorgenoemde *lateral movement* aanvallen, door een experiment uit te voeren dat zulke aanvallen simuleert in een cluster omgeving waar een instantie van GH draait vs. een omgeving waar geen instantie van GH draait.

List of Abbreviations

VM Virtual Machine

K8s Kubernetes

NP Network Policy

SG Security Group

GH Grasshopper

API Application Programming Interface

RCE Remote Code Execution

SA ServiceAccount

KOPF Kubernetes Operator Pattern Framework

NFS Network File System

List of Figures

Table of Contents

TABLE OF CONTENTS

1. Introduction

1.1 Context

Cloud native solutions are a widely adopted approach in the industry for serving highly available, reliable and responsive applications. The cloud native paradigm consists of a set of practices and techniques in modern software development that leverages the advantages of cloud computing environments.[1] It is built upon the concept of multi-tenancy, which enables multiple tenants to make use of the same physical hardware to host their applications. It allows for more efficient resource sharing of the underlaying infrastructure. Therefore, it is more cost-efficient for the cloud provider, which reflects back on cost for the user. Cloud native applications consist of lightweight, independent, and loosely coupled software packages, called microservices. These microservices, run on containers hosted on the Virtual Machines (VMs) of the cloud environment. Because of their lightweight nature, containers can be created more quickly than traditional VMs. This is a key reason they are used over traditional VMs in a cloud setting, considering the fact that they may be created and removed sporadically based on user demand. Additionally, the cloud native paradigm allows for faster development and easier continuous integration for developers.

Containers are managed by a *container orchestrator*. This is a middleware component running on the nodes of the cloud-infrastructure that manages the containers into a cluster.[2] Kubernetes (K8s) is by far, the most widely-deployed orchestrator.[3][4] In K8s, containers are grouped into Pods, which is the smallest unit of deployment in a K8s cluster. Pods consist of logically grouped containers and are connected via the Pod network, which allows all Pods in the cluster to communicate directly to each other. *Network Policies* (NPs) are used to impose rules on said communication. Generally, they are used to isolate different applications and tenants of the cluster, hereby enforcing the principle of least-privilege.

Pods are hosted by nodes of the cloud environment. These nodes are typically Virtual Machines (VMs) running on the cloud environment and communicate with each other over the cloud- or VM-network. *Security Groups* (SGs) are a common way in most cloud environments to enforce security rules upon nodes of the cluster. These consist of a set of rules, specifying allowed ingress or egress communication on a range of ports, using a given protocol. These set of rules - or security groups - then get attached to the nodes, which applies the speciefied rules to the node.

The dominant use of packaging applications as containers, also gives rise to a number of security issues. Containers are generally considered less secure than traditional VMs, because they come with less isolation mechanisms than traditional VMs. This makes them vulnerable to exploits where an attacker can abuse this permissive setting in order to gain access to the underlying node. Recent studies[5][6] have shown that a large portion of container images, contain a high amount of vulnerabilities. These would e.g. enable an attacker to perform Remote Code Execution (RCE) on a given container. These vulnerabilities combined with an escape of the container environment, could potentially allow attackers to gain a foothold of the nodes running in a cloud environment. In this thesis, we will evaluate a post-exploitation

scenario, where an attacker gets this kind of access and then tries to move to other nodes of the cluster.

As mentioned, containers are grouped into pods, which are connected through the Pod network. This is a virtual network connecting all the pods (and containers running inside the pods) to each other. This network is instantiated by a plugin component running on the cluster. This component ensures that routes between pods (and containers) exist. The plugin can achieve this in two ways: either by using an overlay network, where pod packets are encapsulated before they get sent over the VM-network, or by using underlay network. Existing research, refers to this as *native routing*. In this setting, pod packets get sent over the VM-network as-is and the plugin component ensures that corresponding routes are present in the VMs. In a context where applications need to respond fast to user requests - such as low-latency applications - native routing is most commonly used. The reason for this, is that native routing introduces less overhead due to not having to encapsulate every packet on every turn. This leads to a better performance in latency, which is a desired property in an environment where low-latency applications are running.

However, sending container packets trough the VM-network in this way, requires every port and protocol used by container communication, to be opened on the VM-level as well. Even when no containers hosting the application are running on the node. This often leads to cluster administrators, statically attaching a security group that opens up the ports required by the application on every worker node in the cluster. This leads to an over-permissive setting, where the principle of least-privilege is not enforced on the VM-network. Indeed, when a node is not hosting a container of said application, the corresponding ports do not need to be opened on that node. Therefore, this setting - while meeting the latency requirements of low-latency applications - comes at the expense of an increased attack vector on the cluster. Especially in the context of vulnerable containers and potential escapes of the container context, allowing access to the host. In this case, an attacker can easily move to other nodes of the cluster by e.g. opening a shell connection on the allowed ports of the nodes. In current literature, this is more commonly known as *lateral movement*. It is exactly this behaviour we intend to reduce.

Grasshopper [GH] is a tool designed here at KULeuven by Gerald Budgiri. It dynamically configures security groups, based on the network policies and pod placements in the cluster. Essentially, it enforces the principle of least-privilege, specified at the application level, upon nodes of the cloud-network. Hereby reducing the attack surface on the cloud environment. It does so, by continuously monitoring the K8s api-server for pod and policy events and applying the appriopriate changes to corresponding security groups. More specifically, it checks which network policies apply to what pods running on a node. Then, it configures the security groups to allow only the traffic specified by the network policy, restricting all other communication. Important to note, is that respective traffic is only allowed when a pod is actually running on the node. This reduces unnecessary connections and therefore reduces the attack surface on the cloud environment.

1.2 Problem

While existing research shows GH effectively reduces the attack surface on the cloud infrastructure, it has not yet focussed on scalability and reliability of the program. Therefore, is is not yet clear how the program scales in a real-world cluster setting, where a large number of pods and policies may exist and applications may be aggresively auto-scaled, leading to a large number of extra or removed pods. These extra or removed pods all trigger events, in which case the GH program has to potentially modify, create or remove security groups. In the case of aggressive auto-scaling, these events may all be triggered concurrently, creating a lot of work for the GH program to carry out. For all the new or removed pods, the program has to check if policies apply to the given pods and if corresponding security groups need to be reconfigured. This case of aggressive auto-scaling, has not yet been researched in the context of Grasshopper.

Furthermore, the current implementation of the program does not posses certain features, allowing the program to be really reliable in a real-world setting. Those features include, starting up the program in an already active cluster and recovery of the program in case of failure. These are necessary features in a real-world scenario, where failures may be the case. Additionally, allowing the program to start up in cluster environment that is already active, is paramount for allowing adoption by existing organisations.

Additionally, the program is currently designed to run as code on the master node. In order to run the program in an existing cluster setting, manual setup is required. This process often proves to be a difficult experience, which may lead to a large setup time, resulting in large adoption barrier for organisations looking to use the functionality provided by the program.

1.3 Goals

This thesis aims to resolve the issues raised in the previous section. It attempts to make the Grasshopper tool more scalable and reliable, in order to be useful in a real-world cluster setting. To achieve this, we explore a pattern commonly used in the Kubernetes paradigm, called the Operator pattern. In this pattern, a Pod is deployed in the cluster that monitors certain kinds of Kubernetes resources (e.g. pods and policies) and takes appropriate action based on the observed events.

Operator frameworks commonly provide tools for more efficient and robust handling of events. We intend to get a clearer view on the scalability of the current implementation of GH in a real-world cluster setting and aim to improve it by using an Operator approach. In this approach, we implement GH using an existing Operator framework, called the Kuberenets Operator Pythonic Framework (Kopf). This is a Python framework that provides tools to easily watch certain resources (such as pods and policies) in the cluster in a robust and reliable way. Additionally, it offers more efficient handling of events of those resources. We intend to explore how using those tools affects GH's scalability in a real-world cluster setting.

Furthermore, we intend to improve the reliability of the program. We aim to do this, by introducing two features to the program. The first one would allow for the program to start up in an already active cluster. The second one adds the ability to recover from failures. We aim to add these features first for the original implemenation of GH and then for the version that is implemented with the Operator framework. Then, we aim to research to average recovery times of the different versions.

Lastly, since Operators are deployed as pods in the cluster, we intend to build a GH-image that can run by a container. This would allow for a GH pod to be deployed in the cluster, with a single command, which would prevent the potentially large setup time and would allow for easier deployment of the tool in the cluster.

1.4 Approach

In this thesis, we extend the current implemenation of the Grasshopper tool. To achieve this, we first looked at the existing implementation of GH. However, we quickly realised the quality of the code was fairly low, which made it hard to extend. Therefore, we decided to reimplement the baseline version, which lead to our own baseline version (V1). We also added the two aformentioned features to the program, allowing it to restart and start up in an already active cluster. Next, we applied the Operator pattern to the baseline version, which lead to a second version of the program (V2). We then adapted the two aforementioned features, by using the tools provided by the kopf framework. Lastly, we extended the Operator version with a feature that stores the information necessary in a persistent database (V3). This feature is meant to decrease the recovery time of the program, in case of failure, since it can read in all the necessary information, instead of having to reevaluate all the things that happened in the cluster.

Then, we evaluate the three different versions of the GH program (V1, V2 and V3) on several relevant aspects. These aspects include: scalability, reliability and security. In order to evaluate the scalability, we design an experiment that shows how system-usage and latency of event-handling scales in function of different-sized pod creation spikes.

In order to evaluate the reliability, we design an experiment that measures the average recovery times of the different versions. In this experiment we first setup a cluster of varying sizes, then simulate a failure and lastly, let the program recover. We measure the time it takes for the different versions, to be fully up and running again. In this way, we examine how the different versions perform on the recovery aspect, which is the most important aspect of reliability.

Lastly, we show the effectivess of the different versions in improving security. We do this, by reviewing a real-world, post-exploitation scenario where an attacker has gained access to a node in the cluster now tries to move lateraly to other nodes by performing a reverse-shell attack. We show that our different versions effectively stop this movement in the case where nodes should not have open connections.

1.5 Structure of the text

We will now describe how the remainder of this text is structured. In Chapter 2 we give some background information about Kubernetes and Openstack and motivate the need for least-privilege enforcement of network policies on the security group configuration of the cloud platform. In Chapter 3 we describe the GH algorithm and the implementation desgin. Chapter 4 introduces the second version (V2) created of the program, by applying the Operator pattern to it. We implement this version using an exisiting Operator framework, called Kopf. Then, we describe the third version (V3) created of the program, which adds persistence of its vital information to a permanent datastore. In chapter 6, we perform an evaluation of the three versions on their scalability and reliability features and illustrate its effectiveness in reducing lateral movement in a post-exploitation scenario. Lastly, we conclude our findings in Chapter 7.

2. Background and Motivation

This chapter provides some background information of relevant topics for understanding the thesis and motivates the need for least-privilege enforcement of security groups in response to pod scheduling events.

2.1 Openstack

2.1.1 Openstack Introduction

A lot of different cloud platforms exist, such as Amazon Webservices [AWS], Google Cloud, Microsoft Azure and many more. Openstack is the most popular open-source platform, allowing organisations to build their own private cloud environment. [7] Since the platform is open-source, it is also used in a lot of academic endeavors. It consists of a set of tools, installed on physical machines that allows users to manage compute, storage and networking resources. Nodes of the cluster are instantiated by VMs running on the physical infrastructure. These can be created or removed by the provided CLI-tools or the Horizon component, which is Openstack's Web Interface. Additionally, it provides a way of managing communication between the nodes hosted on the platform. In this way, rules can be specified for node-to-node communication. As is the case in most popular cloud platforms, this is achieved through the use of Security Groups.

2.1.2 Security Groups

A Security Groups (SG) is used to impose rules on node-to-node communication. It consists of a set of security group rules, each having the following fields: IP Protocol, Ethertype, IP range, Direction and a Remote Security Group field. Each rule, specifies allowed ingress or egress communication for a certain range of ports on a certain protocol, from or to nodes within a certain IP-address (CIDR) range or from or to a certain Remote Security Group. Security groups get attached to nodes, which applies all its rules to the node. When the Remote Security Group field is specified, all nodes are selected as destination or source, to which that Security Group is attached. An example of an SG can be found in Figure 2.1 . This example shows a security group that allows all egress traffic to all other nodes and ingress traffic only on port 8080 from nodes that have the same security group attached to them. The Remote Security Group field for this rule shows the id associated with the SG.

IP Protocol	Ethertype	IP Range	Port Range	Direction	Remote Security Group
tcp	IPv4	0.0.0.0/0	8080:8080	ingress	f9e66b83-d49f-4bc7-b004-26ddd6e2b0a4
None	IPv4	0.0.0.0/0		egress	None

Figure 2.1: Example SG

2.1.3 Default Setting

The usual way of setting up an Openstack cloud environment, is by attaching a security group to all nodes, opening up all ports and protocols. This security group would set itself as a Remote Security Group, which leads to nodes being able to communicate with all other nodes in the cloud environment. Alternatively, a security group opening up all ports and protocols for the whole CIDR range of the Pod-network, could be attached to all nodes, which would allow the whole Pod-network to communicate on the VM-network. In the case of native routing, this is necessary in order to allow pod traffic to flow over the node network. However, these approaches do lead to an over-permissive setting of the network, since not all ports and protocols have to be open on all nodes. We will discuss this problem in more detail in section 2.3.

2.2 Kubernetes

2.2.1 Introduction to Kubernetes

Kubernetes [K8s] is the most widely used container orchestrator platform. It is used to automate deployments of containerised applications, offering features such as auto-scaling, self-healing and automated rollbacks. The system can be installed on virtually any cloud environment, which is one of the key reasons it is the most widely adopted container orchestrator. Furthermore, it provides networking functionality for pods, allowing them to communicate with each other. It is designed for multi-tenancy, meaning that multiple tenants and their applications can run on the same cluster, while still preserving isolation between those tenants and applications. It achieves this by having the concepts of namespaces, labels and network policies. A namespace refers to a collection of resources in the cluster, while labels annotate certain characteristics of those resources. In this way, network policies can be used to specify rules for pod-to-pod communication, with the pods belonging to certain namespaces and possesing certain labels.

Kubernetes employs the infrastructure-as-code approach, where users of the system define the state they want the cluster to be in through declarative configuration of desired state in .YAML files. K8s is designed to make sure the system is in this desired state. It achieves this by continuously monitoring the cluster and its pods and verifying if their actual state differs from the desired one. Should that be the case, it takes the necessary actions to put the cluster back in the desired state. This process, is also known as the *reconciliation loop*.

A Kubernetes cluster consists of a *control plane* and a *data plane*[8]. The control plane consists of one or more *master nodes* and is responsible for ensuring the proper workings of the cluster. The data plane, in turn, consists of one or more *worker nodes*, which carry out actual work by hosting pods. Each node is managed by the control plane and contains the components necessary to allow pods to run.

The *kube-api-server* is the most important component in the cluster, that runs in the control plane. It exposes a REST api allowing users, components of the cluster or external components

to request or modify the current state of the cluster.

The *etcd* components constitute the database of the cluster. It persistently stores the desired state of the Kubernetes resources, along with their current state.

The *kubelet* can be described as the primary node agent. It communicates with the api-server to report its status and can take request from a scheduler to run pods. It works together with the container runtime to actually run containers.

The *container runtime* is a software component that allows containers to run on the node. It is responsible for starting up and removing containers and reports their status back to the kubelet.

Kubernetes allows the deployment of containerised applications, by allowing containers to run inside Pods, which get scheduled on nodes across the cluser. These pods may need to alter other Kubernetes resources. This can be achieved by using certain endpoints exposed by the api-server. However, some form of authorisation is necessary. For this reason, K8s has introduces the notion of ServiceAccounts.

A ServiceAccount (SA) is a K8s resource that has certain permissions associated with it. An example of such a permission could be: the ability to request pod information or to modify pods in the cluster, or to request the current network policies in the cluster. A pod needing these permission, would specify the corresponding SA in its .yaml file. The K8s system provides the ServiceAccountToken to the pod upon startup. The pod uses this token to authorize for its actions, when making requests to the api-server. A ServiceAccountToken, is a specific kind of Secret, used to store a credential that is associated with a ServiceAccount.

A Secret is a K8s resource, containing some sensitive information. All Secrets are stored in the etcd component, which are stored unencrypted by default. However, they can be configured to be stored with encryption. It allows to seperate sensitive information from pod .yaml files and container images, which reduces the risk of the secret being leaked.

2.2.2 Kubernetes Networking

Kubernetes defines a flat network model, which states that every pod should be able to communicate with every other pod in the cluster, without relying on Network Address Translation [NAT]. This means that every Pod in the cluster gets its own unique IP address, which can be used to directly target other pods. The K8s system delegates its networking functionality to a Container Networking Interface [CNI] plugin. This plugin achieves Pod-to-Pod communication, by making sure routes between pods exist on the nodes.

There are generally two ways in which this can be achieved: either by using an overlay network, where pod-packets get encapsulated before traveling to another node. Or by using an underlay network approach, more commonly known as native routing. When this approach is used, pod packets travel over the VM-network without any encapsulation. The CNI-plugin commonly uses the Border Gateway Protocol (BGP) to advertise routes for pods that reside on other subnets. In this way, overhead from having to encapsulate packets on every node hop is prevented, which allows for better latency performance. This is a desired property in clusters hosting low-latency applications.

In order for this to work, the IP-addresses, ports and protocols used by the pod packet, have to be allowed on node-to-node communication as well. This means that every IP-address, port and protocol that is used by the entire pod network, have to be open on the VM-network as well. This is because every pod traffic has to be able to travel over the node network. Unfortunately, this introduces an increased security risk, since this leaves the node network in an over-permissive setting. Indeed, even when nodes do not host pods for which the ports and protocols need to be opened, they still will be, which leads to the VM-network not being in a least-privilege setting.

The CNI plugin, also enforces security rules for pod-to-pod communication, defined in the network policies.

2.2.3 Network Policies

Network policies are used to impose rules on pod-to-pod communication. As mentioned, K8s introduces the notion of namespaces and labels. Namespaces are used to group over a collection of resources. Labels are key-value pairs attached to resources, annotating them with a certain characteristic. A relevant example could be the key-pair `"role":"database"`. being attached to a pod, stating that the pod is running a database related microservice.

Network policies are specified in .yaml files, of which the spec field defines the actual desired state of the NP.

The `spec` field consists of a `podSelector` and an `ingress` and/or `egress` field. The podSelector selects a group of pods by means the use of labels. This selects all pods that having matching labels, meaning that that specified labels are at least a subset of the labels of those pods. For those selected pods it now defines ingress and/or egress rules.

Ingress and egress rules have respectively a `from` and a `to` field, in which the following sub-fields may be present. A `namespaceSelector`, `podSelector` and/or an `ipBlock` field. The namespaceSelector is used to select certain namespaces of pods, while the podSelector selects pods based on their labels in the namespace of the current network policy. Additionally, a CIDR range of pods can be specified by use of an `ipBlock` field. This selects sending or receiving pods based on their IP-address.

An example network policy can be found in Figure 2.2 . It selects Pods with labels: `"role":"frontend"` in the namespace `app-a`. It then defines for those Pods an ingress and egress rule. The ingress rule specifies the selected Pods can receive traffic from all Pods with the labels `"role":"backend"` on port `80` using the TCP protocol. The egress rule specifies

that the selected pods can send traffic to pods with labels `"role":"backend"` on port `80` using the TCP protocol. Essentially, the Network Policy states that frontend Pods in the namespace app-a can communicate in both direction with backend Pods on the standard HTTP protocol.

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-network-policy
  namespace: my-app
spec:
  podSelector:
    matchLabels:
      role: frontend
  ingress:
    - from:
        - podSelector:
            matchLabels:
              role: backend
          ports:
          - protocol: TCP
            port: 80
  egress:
  - to:
      - podSelector:
          matchLabels:
            role: backend
        ports:
        - protocol: TCP
          port: 80
```

Figure 2.2: Network Policy Example .YAML file

2.3 In-depth enforcement of firewall rules

We have discussed the two layers of communication in a cluster running on top of a cloud platform. On the cluster layer, pods communicate to each other and network policies are defined to provide isolation between pods belonging to different tenants or applications. Cluster administrators are concerned with isolation on this layer. On the cloud layer, node-to-node communication happens and security groups are defined to specify rules for this communication. Cloud administrators are concerned with isolation on this level. However, isolation is usually defined on the application-level, which means that we consider network policies to be the ground truth. Network policies provides a mechanism to isolate traffic between pods that are running different applications. However, in the case of container vulnerabilities and potential escapes, this mechanism is not sufficient. In this scenario, an attacker gains access to the underlying node, in which case the network policy rules can be omitted entirely. This is because the attacker can send network packets directly from the nodes and therefore not the network policies apply, but the security groups defined on the VM-level.

In the case where the cluster is hosting low-latency applications, native routing is most likely used to enable per-pod routing. In this setting, cloud administrators often statically attach a security group to all worker nodes, opening up ports and protocols that are required by the application. This is necessary in order to allow the application to be fully operational in the entire cluster. However, it is possible that pods of the aforementioned application are not hosted on certain nodes. In this case, we do not want those ports to be opened in order to decrease possible lateral movement attacks. Indeed, in the latter case, those open ports are not necessary for the application to function and only increase the attack surface in the case of exploited containers and escapes from the container context.

Possible mitigations for these lateral movement attacks, could be to divide cluster resources and specifically assign them to an application. Then, security groups could be statically attached to isolate application-specific regions. However, this would decrease the ability to efficiently share resources.

Therefore, it would be desirable if security groups would be dynamically configured, taking into account the applied network policies and placements of pods on nodes. In this way, only when pods are actually hosted on nodes, to which certain network policies apply, will the corresponding security groups be modified to allow that pod traffic on those nodes as well. Doing it in this way, would truly enforce the principle of least-privilege - specified in the network policies - upon the VM-level, hereby significantly reducing the attack surface in the case of escapes.

This dynamic modification of SGs is exactly what Grasshopper is designed to do. It enforces the principle of least-privilege on the VM-layer, by taking into account network policies and pod placements. Therefore, only connections between nodes that are truly necessary for the application to function properly, will be configured. All other connections, will be prevented. It achieves this by watching pod and network policy resources in the cluster and configuring security groups accordingly.

2.4 Operators

2.4.1 Operator Pattern

An Operator is a controller for an application-specific resource in a Kubernetes cluster. A controller is a K8s component that watches a resource of the cluster, through the K8s api-server and ensures that it is in its desired state (defined in the resource's .YAML file).

An Operator is a custom controller, that is added to the K8s system and is responsible for managing a certain (custom) resource. These resources can be defined through the use of Custom Resource Definitions (CRDs). They are themselves a Kubernetes resource that can be applied in the system. When these are applied, resources of this new type can be created. An Operator can then manage these resources. This is the most common way in which an Operator is used. However, the managed resources do not necessarily have to be Kubernetes resources. They can also be resources defined in the application domain, such as Security Groups. A visual diagram of an Operator can be found in Figure 2.3, which originates from the white-paper on Operators, written by the Cloud Native Computing Foundation (CNCF).[9] This diagram shows the Controller (or Operator) watching several things in the cluster. Which kind of things an Operator watches specifically, depends on the case. However, in any case, it at least watches a resource within the cluster and takes appropriate action when events of this resource come in. It then makes sure the managed resource is in its desired state. The desired state does not necessarily have to be defined in the Kubernetes system itself. This can also be something defined in the application domain and it can be something abstract, such as whether or not, the events of the watched resources have been successfully handled or not.

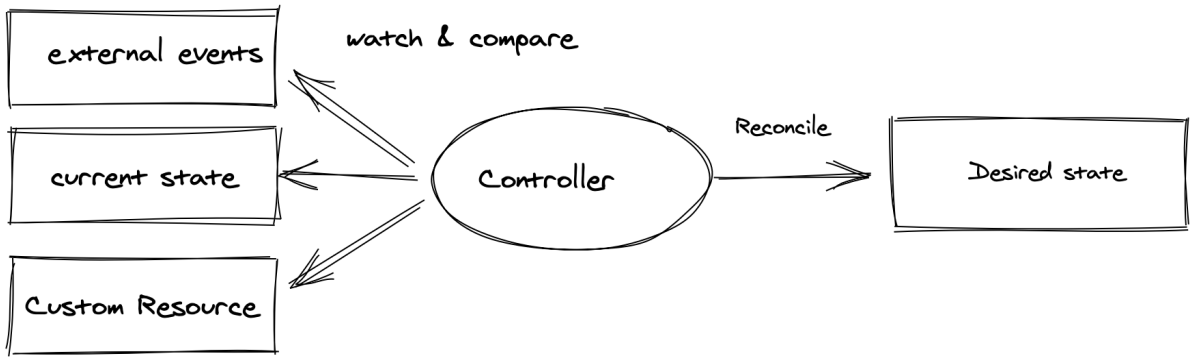

Figure 2.3: Operator Diagram

According to this specification of an Operator, we can view GH as a kind of Operator. In this case, the resources that are being watched are pods and network policies. The desired state can be expressed as the least-privilege configuration of Security Groups, in response to pod placements and network policies applied in the cluster. When events of pods and policies come in, the GH Operator makes sure that Security Groups are configured appropriatly, so that they enforce the principle of least-privilege in the cloud layer.

Additionally, the white-paper defines some capabilities an Operator should have. The most important and relevant ones for this thesis, are the backup and recovery features. According to the document, an Operator should have the functionality of automatic backups and the ability to recover from failure. We will implement these features in our V3 version, hereby applying (a part of) the Operator pattern to the GH program.

2.4.2 Operator Frameworks

In order to implement the Operator pattern, there are a lot of Operator frameworks available, which allow for easy creation of a custom Operator. Examples of such frameworks are: CNCF Operator Framework, Kubebuilder Kopf [10], and Operator SDK. These all offer a specific set of tools and features. We have chosen to use the Kopf framework, because it is written in Python, which nicely intergrates with our current version and allowed us to easily apply the pattern to our existing work. The main goal of these frameworks is to ease the creation of an Operator and to offer tools for robust and efficient handling of events.

The Kubernetes Operator Pythonic Framework (Kopf) offers a way of efficiently handling resource events, by allowing the handlers the run concurrently. It achieves this by creating a worker for each watched kubernetes resource. These worker consume the events of a said resource and run in parallel to each other, whichs allows for concurrent processing of events. Events of a single resource are handled sequentially in order to ensure consistent state.

We will leverage this feature in our V2 version in order to improve the efficiency of event handling, therefore improving scalability of the program.

2.5 Related Work

This section describes some previous work, related to Grasshopper.

2.5.1 NFVGuard

Network Functions Virtualization (NFV) is an architectural networking concept in which physical networking infrastructure is virtualized, allowing network service providers to create virtual networks on demand, thereby improving scalability in a cost-effective way. It enables multi-tenancy of network services by allowing them to share the same physical infrastructure while maintaining isolation between them. The NFV stack consists of four layers: the Service Orchestration layer (L1), the Resource Management layer (L2), the Virtual Infrastructure layer (L3), and the Physical Infrastructure layer (L4). The instantiation of this stack can be complex and can consist of multiple entities across the various layers. All of these layers define a set of security properties that must hold. However, due to the complex nature of the NFV stack, inconsistencies might exist between the properties of these layers. These vulnerabilities may be exploited by adversaries to attack the system. NFVGuard aims to ensure that the security of each layer is consistent with the other layers. [11] It does this by verifying the security properties for one layer and then ensuring consistency with adjacent layers.

As will be clear from the next chapter, the security verification of a single layer can be related to the Grasshopper program's verification of network policies at the cluster layer. The consistency verification between different layers can be seen in the translation of network policies to security group configuration.

NFVGuard relates to the Grasshopper domain in that both involve virtualized networks operating across different layers, where the security properties of one layer need to be consistent with the security properties of adjacent layers. However, Grasshopper is concerned only with the cluster and cloud layers.

2.5.2 Kano

Kano is a tool designed for efficient verification of container-level network policies in a Kubernetes cluster. [12] As is the case in Kubernetes, it is designed to work with label-based annotations of the containers and network policies being expressed through the use of such labels. It works by creating a bitwise reachability matrix, in order to model the bipartite graph of the container network, which it leverages to achieve an efficient verification operation. This is necessary because of the possible large amount of containers running in a cluster and frequent updates of network policies.

The verification of network policies refers to ensuring that they do not violate the network constraints, as this can lead to vulnerabilities in the cluster or availability issues of certain containers. Kano identifies a number of situations, in which network policies violate the network constraints. Such violations are: a container is reachable by all other containers, a container is not reachable by any container, a container can reach another user's container, connections allowed by the policy are already allowed by existing policies and a policy conflicts with another policy. As will become clear in the next chapter, these last two situations (redundancy and conflict) will be checked by Grasshopper as well, when a new network policy is applied in the cluster.

In order to ensure a secure and well running cluster, these situations need to be avoided. Verification of the security rules in the cluster layer is therefore an important aspect of achieving a secure system. In addition to policy verification, Grasshopper adds the consistency verification between cluster-level policies and cloud-level security groups as well, while Kano is only concerned with the verification of security properties at the cluster-level.

2.5.3 Bastion

Bastion is located in the landscape of modern-based container platforms, where containers are running on hosts using the shared kernel-resource model, making it hard to provide proper isolation between the containers and their hosts. [13] Additionally, container communication relies on the host OS network stack and virtual networking features, which gives rise to a number of security challenges, such as the loss of container context of packets and unrestricted host access. These security vulnerabilities may lead to communication of containers with other containers or hosts that should not be allowed.

Bastion addresses these problems by enforcing the principle of least-privilege regarding this communication. It does so by gathering policy and network information from active containers and building an inter-container dependency map. This map is built by the manager component and is used to deny access to containers or hosts which are deemed irrelevant. This enforcement is done by a per-container network stack that gets deployed by the manager component.

Bastion is concerned with ensuring least-privilege in communication between containers and hosts or other containers at the host level. Unlike GH, it addresses the problem of network-based attacks originating directly from containers, where privilege escalation is not required. In contrast, GH focuses on container escape scenarios, which requires it to ensure least-privilege at the cloud layer.

3. Grasshopper

This chapter is used to describe and explain the Grasshopper algorithm. As mentioned, Grasshopper is a tool designed by Gerald Budgiri in his doctoral thesis here at KULevuen. Its purpose is to dynamically enforce a least-privilege setting of security group configuration in a cloud platform, in response to pod placements and network policies defined in a Kubernetes cluster. This chapter explains how it functions in a Kubernetes cluster running on top of the Openstack cloud platform. Yet, its design allows for integration with any cloud platform offering an API for security group modification.

This chapter reviews the GH algorithm, its components and its implementation design. We start off with the some preliminaries. In the section that follows we introduce some formalisms that allow us to accurately explain the algorithm and how it is implemented by its components. We then present the system's architecture and its key components. Following this architectural overview, the algorithm that is implemented by these components is examined.

This work is heavily inspired by Gerald, the design and implementation of this version was done in collaboration with Robbe Serry.

3.1 Preliminaries

Goals

The first goal of the program is *correctness*. This means the following: if two pods are allowed to communicate to each other on a given port and protocol, this communication should also be allowed on the nodes hosting the aforementioned pods.

The second goal is *least-privilege*, meaning that traffic between nodes should only be allowed when there are pods running on them and there exists a network policy that selects one of them and has an allow rule that refers to the other, which allows the same traffic.

In essence, the goal of GH is to respond to pods being scheduled in the cluster, while taking into account the network policies that are in place. In its response, it should configure security groups in such a way that, if two pods are allowed to communicate in the cluster, the nodes that host these pods should be allowed to communicate as well. It should therefore take notice of the policies that are applied and the nodes on which the pods are being scheduled. When no pods that are subject to a network policy permitting specific communication, are scheduled on a node, such traffic should not be allowed on that node.

Network Policy Verification

As NPs are considered the ground-truth in our approach, it is important that these policies themselves are not misconfigured. Therefore, when first processing a new policy being applied in the cluster, GH checks it for conflicts, redundancy or just too overly broad access permissions. We briefly explain these misconfigurations.

- A *conflict* happens when a network policy weakens an exsisting policy. This can happen when it allows traffic to more labelsets or Pod ranges for the same select labelset or when it allows the same traffic for a subset select labelsets.
- A *redundant* network policy is one that allows communication between pods, that is already completely allowed by existing policies in the cluster.
- *Too overly broad access permissions* refers to policies that select all possible labelsets in the cluster.

PNS vs PLS

Grasshopper is able to achieve least-privelege security group configuration in two different manners. In the first manner, a dedicated security group is attach to each node in the cluster. Then, security group rules are added to them dynamically that represent the communication allowed by network policies between pods and the pods running on the nodes subject to those policies. This variant is called the Per Node Security group variant (PNS).

The second manner to configure security groups is called the Per Labelset Security Group variant (PLS). In this approach, a security group is created for each labelset that is used in a network policy, either in the select section or in one of its the allow rules. It then attaches these SGs to nodes that have pods running on them, which are subject the policy's select or allow section. The labelset's security group will consist of rules that mirror the traffic allowed by network policies that reference this labelset.

We will now explain the algorithm that GH uses, in order to accomplish the least-privilege security group setting, in response to pod schedulings and network policies that are applied in the cluster.

3.2 Formalisms

We start by introducing some formalisms.

- A LabelSet refers to a set of key-value pairs. Both the key and value are strings.

$$\text{LabelSet} = \{(k, v) \mid k, v \in \mathit{String}\}$$

- A Pod in the cluster can be represented as a record containing a *name*, a *labelset* and a *node* field. The name field refers to the pod's name in the cluster. The labelset refers to the set of labels that have been assigned to the pod in the Kubernetes .yaml file. The node field refers to the node on which the pod has been scheduled.

$$\begin{aligned}\text{Pod} = \langle\, &\text{name} \in \mathit{String};\\ &\text{labelset} \in \mathit{LabelSet};\\ &\text{node} \in \mathit{Node}\,\rangle\end{aligned}$$

- A Node is a record containing only its name.

$$\text{Node} = \langle\, \text{name} \in \mathit{String}\,\rangle$$

- A Network Policy is represented as a record containing a *name*, a *select* field and an *allow* field. The select field is a LabelSet, specifying which pods it selects. This means that the policy applies to all pods, which labelsets are a superset of this policy's select field. The allow field is a list of tuples specifying traffic that is allowed to or from pods or IP ranges. It contains either a LabelSet or a CIDR value as the first element and a Traffic instance as the second element.

$$\begin{aligned}\text{NetworkPolicy} = \langle\, &\text{name} \in \mathit{String};\\ &\text{select} \in \mathit{LabelSet};\\ &\text{allow} \in \mathit{List}(\mathit{LabelSet} \cup \mathit{CIDR} \times \mathit{Traffic})\,\rangle\end{aligned}$$

- A CIDR record is used to represent an IP range.

- A Traffic record is meant to represent the traffic that is allowed to or from the LabelSet or CIDR instance. It consists of a *direction*, *port* and *protocol*. A direction is either a "INGRESS" or "EGRESS" value. The port is an integer, specifying the allowed port of the allowed trafic, while the protocol specifies the allowed protocol. This is either a "TCP" or "UDP" value.

$$\begin{aligned}\text{Traffic} = \langle\, &\text{direction} \in \{\text{INGRESS}, \text{EGRESS}\};\\ &\text{port} \in \mathit{Integer};\\ &\text{protocol} \in \{\text{TCP}, \text{UDP}\}\,\rangle\end{aligned}$$

- A Security Group can be formalized as a record, containing a *name* and a *rules* field, which is a set of *Rule* instances. These rules define the allowed behaviour of the Security Group. When attached to a node, this set of rules apply to the traffic being sent to or from the node.

$$\begin{aligned}\text{SecurityGroup} = \langle\, &\text{name} \in \mathit{String};\\ &\text{rules} \in \mathcal{P}(\mathit{Rule})\,\rangle\end{aligned}$$

- A Rule refers to a security group rule and consists of a *target* field and a *traffic* field. The target field is either a Security Group or a CIDR value, refering to the "target" of the traffic allowed by the rule. The traffic field specifies which *Traffic* is allowed by the rule.

$$\begin{aligned}\text{Rule} = \langle\, &\text{target} \in \mathit{SecurityGroup} \cup \mathit{CIDR};\\ &\text{traffic} \in \mathit{Traffic}\,\rangle\end{aligned}$$

3.2.1 Main Component: ClusterState

The main component of the GH algorithm is the ClusterState. It is used to store all the information necessary for the program to successfully modify security group configuration, according to the network policies in place and the pods that are scheduled on nodes in the cluster. It stores the *nodes* of the cluster, the *pods* that are scheduled on it, the *policies* that are in place, the *security groups* that are created, and policies that did not pass the verification check. These

policies are stored in its *offenders* field. It also contains a *map* field, which constitutes the foundation for the algorithm. This map field stores a mapping of *LabelSet* instances, to *MapEntry* instances.

$$\begin{aligned}\text{ClusterState} = \langle\, & \text{pods} \in \mathcal{P}(\mathit{Pod});\\ & \text{nodes} \in \mathcal{P}(\mathit{Node});\\ & \text{policies} \in \mathcal{P}(\mathit{Policy});\\ & \text{securityGroups} : \mathit{String} \to \mathit{SecurityGroup};\\ & \text{offenders} \in \mathcal{P}(\mathit{Policy});\\ & \text{map} : \mathit{LabelSet} \to \mathit{MapEntry}\,\rangle\end{aligned}$$

A MapEntry is a record consisting of a *selectPols* field, an *allowPols* field and a *matchNodes* field. The selectPols field refers to all policies that use the labelset in its select section. The allowPols field refers to all policies that have an allow rule, where the given labelset (used as the key in the map) is being used as an allowed sender or receiver. The *matchNodes* field refers to all nodes that have a pod running on them where the given labelset (used as the key in the map) matches the labelset of the pod. A labelset L matches a pod's labelset L_p when it is a subset of this pod's labelset. This means that the labelset's key-value pairs all exist in the pod's labelset and that if the labelset L belongs to a policy's select or allow section, that the behaviour defined in this section applies to the pod.

$$\begin{aligned}\text{MapEntry} = \langle\, & \text{selectPols} \in \mathcal{P}(\mathit{Policy});\\ & \text{allowPols} \in \mathcal{P}(\mathit{Policy});\\ & \text{matchNodes} \in \mathcal{P}(\mathit{Node})\,\rangle\end{aligned}$$

$$\text{matching}(L, \text{pod}) \iff L \subseteq \text{pod.labelset} \quad (L, \text{labelset} \in \mathit{LabelSet})$$

When dealing with new a pod, GH can consult this map to easily find out which policies select it or which policies use it in an allow-rule. Furthermore is has instant access to the *matchNodes* field, which effectively stores all nodes matching this labelset. It can use all of this information to know, to which policies the pod is subject, and on which nodes the pods that are refered to in the allow or select section of those policies, are located. Therefore, it has all the knowledge required to allow traffic between nodes that is allowed by the network policies in the cluster in a least-privilege fashion, both in the PLS and PNS variant. Important to note, is that we assume a *DenyAll* policy is in place, which by default denies all trafic that is not explicitly specified in a network policy.

3.3 Components & Design

In this subsection, we discuss GH's main components and how they relate to each other. The components we will discuss are: the *Watcher*, the *WatchDog*, the *Matcher* and the *SecurityGroupModule*. An overview of the components and their relation to each other can be found in Figure 3.1

- The *Watcher* monitors pod and policy events using Kubernetes' API server watch mechanism. This mechanism establishes a persistent HTTP connection that continuously

streams all pod and network policy events to the component. Upon receiving an event, it invokes the corresponding handler function in the *WatchDog* component.

- The *WatchDog* component is responsible for verifying network policies and updating the ClusterState's relevant datastructures. When a new pod comes in, it adds it to the the *ClusterState.pods* field and update's the *matchNodes* field with the pod's node for every matching labelset in the cluster. When a new policy is added to the cluster, it updates the *matchNodes* field for every node that has a pod matching the policy's select labelset and for the labelsets of its allow rules.
- The *Matcher* component is responsible for converting the information in the ClusterState's *map* into an appropriate security group configuration, allowing all communication allowed by the network policies that are applied in the cluster. This can happen in two ways, either in the PNS way or the PLS way. Therefore, the *Matcher* component has two different implementations of its methods. In order to actually modify the security group's configuration, the *Matcher* invokes the SecurityGroupModule.
- The *SecurityGroupModule* provides all functionality required to manage security groups, including creating, attaching, detaching and removing security groups, as well as adding and removing rules to them. The functionality needed for this component is dependent on the mode GH is running in. Therefore, this component has two different implementations as well, one for the PNS variant and one for the PLS one.

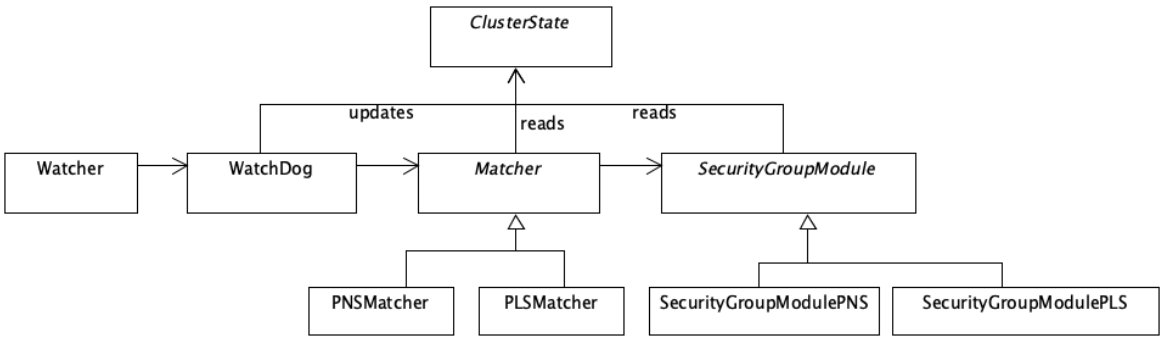

Figure 3.1: Grasshopper Design

3.4 Algorithm

This section dives deeper into the algorithm that is implemented by the various components. We will examine the methods used by these components, when handling pod and policy events. For the sake of brevity, we will only describe the methods used by the PLS variant. However, the flow of the program stays the same for the PNS variant, in which only the functionality of the Matcher and SecurityGroupModule components differs. We will describe in detail the case for when a new network policy is applied in the cluster and for when a new pod is created.

Handling a new policy event

In this scenario, a new network policy is applied in the cluster and the K8s api-server streams its event to the Watcher component. We will now describe step by step what happens in each component.

1. The Watcher receives the policy event and checks if its event type is "CREATED", hereby calling the Watchdog's HandleNewPolicy(pol) method.

2. The Watchdog verifies the policy and adds it to the ClusterState's offenders if it is misconfigured. Otherwise, it splits the policy (as seen in Algorithm 2) and adds each subpolicy to the ClusterState.*policies*. [1] For each node that has a pod running on it where subpolicy's select labelset matches the pod's labelset, it updates the labelset's mapentry.*matchNodes* with that node. It does the same thing for each node that has a pod running on it where the subpolicy's allow rule refers to a labelset and this labelset matches the pod's. Lastly, it calls the Matcher's SGConfigNewPol(spol) with the subpolicy. The pseudocode for this algorithm, carried out by the Watchdog can be found in Algorithm 1.

3. The Matcher checks if the subpolicy's select labelset L_s has nodes with at least one pod that matches this labelset L_s . When it does, it calls the SecurityGroupModule's AddSg(L_s) with this labelset. When the allow rule of the subpolicy refers to a *LabelSet* L_a it calls the SecurityGroupModule's AddSg(L_a) with this labelset as well. The pseudocode for this method is shown in Algorithm 3

4. The SecurityGroupModule's AddSg(L) method creates a security group for the labelset and attaches it to each node that hosts pods matching the labelset.

[1] Note that in our design a subpolicy is just a *Policy* with only 1 allow rule in its *allow* field.

Algorithm 1 Handle New Policy (Watchdog)

```
1: function HandleNewPolicy(policy)
2:     verified ← verifyPolicy(policy)
3:
4:     if not verified then
5:         add policy to ClusterState.offenders
6:         return
7:     end if
8:
9:     add policy to ClusterState.policies
10:                                          ▷ Update matchNodes for select labelset
11:    for spol in split(policy) do
12:        for node ∈ ClusterState.nodes do
13:            if node hosts a pod where spol.select matches with that pod then
14:                ClusterState.map[spol.select].matchNodes.add(node)
15:            end if
16:        end for
17:
18:                                          ▷ Update matchNodes for allow labelsets
19:        if if spol.allow[0][0] is LabelSet then
20:            for node ∈ ClusterState.nodes do
21:                if node hosts a pod where spol.allow[0][0] matches with that pod then
22:                    ClusterState.map[spol.allow[0][0]].matchNodes.add(node)
23:                end if
24:            end for
25:        end if
26:        Call matcher.SGConfigNewPol(spol)
27:    end for
28:
29: end function
```

Algorithm 2 Split Policy into Sub-policies

```
1: function split(policy)
2:     subPolicies ← []
3:     for allowRule in policy.allow do
4:         subPolicy ← new Policy(policy.name, policy.select, [allowRule])
5:         subPolicies.append(subPolicy)
6:     end for
7:     return subPolicies
8: end function
```

Algorithm 3 Security Group Configuration for New Policy (PLS) (Matcher)

```
1: function SGConfigNewPol(spol)
2:     if |ClusterState.map[spol.select].matchNodes| > 0 then
3:         Call SecurityGroupModule.AddSg(spol.select)
4:
5:         if spol.allow[0][0] is LabelSet then
6:             Call SecurityGroupModule.AddSg(spol.allow[0][0])
7:         end if
8:         sg ← ClusterState.securityGroups[spol.select]
9:         rule ← SecurityGroupModule.ruleFrom(spol)
10:        Add rule to security group.
11:    end if
12: end function
```

The SecurityGroupModule's ruleFrom(spol) method creates a corresponding security group rule from a given subpolicy. It takes the spol.*allow*[0][0] as a *target* for the rule and the spol.*allow*[0][1] as the *traffic* field for the new rule. This rule is then added to the corresponding security group, hereby mirroring the traffic that is allowed by the policy rule.

Handling a new pod event

In this scenario, a new pod is created in the cluster and the K8s api-server streams its event to the Watcher component. We will now describe step by step what happens in each component.

1. The Watcher receives the pod event and checks if its event type is "MODIFIED" and if the *spec.nodeName* field has a value, indicating the pod has been assigned to a node, after which it calls the HandleNewPod(pod) method.

2. The Watchdog adds the pod to the ClusterState.*pods* and iterates through all the labelsets in the cluster (all key values in ClusterState.*map*) to find the ones that match the pod's labelset. For all matching labelsets L, it updates the corresponding mapentry's *matchNodes* field with the pod's *node* and calls Matcher's SGConfigNewPod(L, pod.node) method to carry out the security group configuration accordingly. The pseudocode for the this method is shown in Algorithm 4.

3. The SGConfigNewPod(L, pod.node) method in the Matcher calls the SGConfigNewPol(pol) (see Algorithm 3) method for each policy pol in the pod's labelset's *selectPols* or *allowPols* and attaches the security group of the its labelset to the pod's node. The pseudocode for the this method can be found in Algorithm 5.

4. The SecurityGroupModule calls the Openstack API to carry out the attachment of the security group to the node.

Algorithm 4 Handle New Pod (Watchdog)

```
1: function HandleNewPod(pod)
2:     Add pod to ClusterState.pods
3:
4:     for labelSet ∈ filter(λL: matching(L, pod), ClusterState.map.keys()) do
5:         mapEntry ← ClusterState.map[labelSet]
6:
7:         if pod.node ∉ mapEntry.matchNode then
8:                                         ▷ Pod is the first pod on node to match labelSet
9:             Add pod.node to ClusterState.map[labelSet].matchNodes
10:            Call matcher.SGConfigNewPod(labelSet, pod.node)
11:        end if
12:    end for
13: end function
```

Algorithm 5 Security Group Configuration for New Pod (PLS) (Matcher)

```
1: function SGConfigNewPod(labelSet, node)
2:     for policy ∈ ClusterState.map[labelSet].selectPols ∪
3:       ClusterState.map[labelSet].allowPols do
4:         Call SGConfigNewPol(policy)
5:     end for
6:
7:     sg ← ClusterState.securityGroups[labelset]
8:     if sg ≠ null then
9:         Attach sg to node
10:    end if
11: end function
```

3.5 Concerns & limitations

As can be seen from the Watcher component, the events are streamed in continuously from the K8s api-server, in which case the corresponding handlers for the events are called. However, this happens in a synchronous manner. This means that all events are handled sequentially by the system. This could cause some event-handling latency issues, since the handlers eventually call an Openstack API function when a new pod or policy is added to the cluster. These calls could take up to 2-4s, which can add a significant amount of latency to the event-handling process. Especially in the case where a lot of events may be triggered, for example when pods are aggresively auto-scaled, could this lead to a significant amount of time untill some pods are handled. In this case, there may be application latency, since the pods may be in a ready-state on the nodes, but the security group configuration is not yet in place to allow the necessary communication as required by the application.

Another limitation of the current implementation of V1, is the fact that it does not yet posses the ability to deal with namespaces of pod and policy resources. The current implementation ignores this fact and deals with them as if there is only one namespace in which they all live. This is something that should still be added to the implementation. Allthough, simulation of

namespaces can be done by giving all resources which you would consider to be in the same namespace, a specific label, such as e.g *'app': 'app-a'*.

4. Grasshopper Operator

This chapter describes how the Operator pattern is applied to the orignal GH version (V1), resulting in the GH Operator version (V2), which is meant to improve the scalability and reliability of V1. This version will be implemented using the Kopf framework. In the sections that follow, we will explain how this version works and how its design is related to the original version. First, we discuss the Kopf framework, its desgin and main features that will be used in this version. Then, we discuss the implications of these features and the adaptations that are necessary in order for this version to work. Next, we explain how we expect this version to benefit the scalability of the tool. Lastly, we will briefly discuss the potential security implications of using this approach.

4.0.1 Kopf framework

The Kubernetes Operator Pythonic Framework (Kopf) is a Python framework, designed to easily create Kubernetes Operators. [10] Its main goal is to bring domain-driven design to the infrastructure level. It provides a declarative way to registrate handlers for specific Kubernetes events using Python decorators. In this way, handlers can be easily attached to specific Kubernetes events, such as for example, the creation of a network policy in a certain namespace, or the change in a *spec.nodeName* field of a pod. As can be seen from the previous chapter, this is something that is valuable for our tool and provides a much simpler and cleaner way to deal with Kubernetes events.

Moreover, the framework writes the status of the handlers on the resouce's *.status* field, allowing access to information such as which handler was executed on which resource at what time, and whether or not is was succesful in doing so or resulted in an error. This information is especially useful for the recovery feature we will discuss in one of the following subsections.

4.0.2 Concurrency model

The most important feature of the Kopf framework, is the way it deals with events. It is designed with a resource-based concurrency model in mind, which fundamentally changes how Kubernetes events are processed compared to the sequential approach used in the original Grasshopper implementation. Kopf creates a dedicated worker for each individual Kubernetes resource instance, that are able to run in paralel. [14] This means that when multiple network policies or pods exist in the cluster, each resource gets its own execution context, allowing handlers to process resource events concurrently. While events of a single resource are still handled sequentially to maintain consistency, different resources are handled in parallel. We will leverage this feature in this version, in order to improve the scalability of the Grasshopper tool.

4.0.3 Concurrent handling of pod and policy events

While Kopf's concurrency model, at first sight, seems to have the potential to significantly improve Grasshopper's scalability, we must first look at its original design, which was not created with concurrency in mind. When just calling the WatchDog's handlers for when a new pod is scheduled on a node or for when a network policy is applied in the cluster, there might arrise consistency issues or race conditions. For example, when a new pod is scheduled on a node the Watchdog calls the Matcher to configure the SGs appropriatly for each matching labelset in the ClusterState. Handling two pods that overlap in their respective matching labelsets concurrently, might lead to race conditions and an inconsistent state of the ClusterSate (and Openstack resources). Consider for example the following case: two pods with the same labelset are scheduled on the same node (two equal labelsets are also matching). When these two pods are handled concurrently, the SecurityGroupModule might create duplicate security groups, or duplicate rules, which leads to an inconsistencies in the ClusterState and might leave dangling security groups, when pods get deleted again. This would result in an overpermissive setting of the cloud layer's network.

4.0.4 Formalisation of concurreny problem

In this subsection we clearly identify the situations in which race conditions or inconsistencies might occur. As seen from the previous example, handlers that touch the same labelsets, running concurrently, might lead to inconsistencies in the ClusterState. A handler *touches* a labelset, if it accesses the labelset's mapEntry in the ClusterState.*map*. Therefore, it is clear that no two handlers should run in parallel, that have an overlap in the labelsets they touch.

We now review the cases for when a new policy is applied in the cluster or when a new pod is scheduled on a node, to identify which labelsets the corresponding handlers *touch*.

1. **Handling a new policy event:** For each labelset L in the policy's *select* or *allow* section, the Watchdog updates the labelset's *mapEntry* and calls the matcher's SGconfignewpol function. Therefore, this handler touches the policy's select labelset and every labelset in its allow rules.

   We can formalize the labelsets this handler (for a policy) touches as follows:

   For a policy P with select labelset L_s and allow rules containing labelsets $L_{a1}, L_{a2}, \ldots, L_{ak}$:
   $touchedLabelsets(P) = \{L_s\} \cup \{L_{a1}, L_{a2}, \ldots, L_{ak}\}$

2. **Handling a new pod event:** This case is slightly more complex. When the WatchDog handles a new pod, it adds the pod to ClusterState.*pods*, updates the ClusterState's *map*[labelset].*matchNodes* field and calls the Matcher's SGconfignewpod(pod) for each matching labelset. The SGconfignewpod(pod) calls the SGconfignewpol(pol) for every policy in the labelset's mapEntry's selectPols and allowPols. This method accesses the ClusterState.*map* for the policy's select and allow labelset, therefore also touching these labelsets. So, it touches: (1) all labelsets that match the pod's labelset, and (2) for each policy associated with those matching labelsets, it also touches the policy's select labelset and all labelsets from the policy's allow rules.

   We can formalize the labelsets this handler touches as follows:

   For a pod with labelset L_{pod}, let M be the set of all labelsets matching L_{pod}: $M = \{L \in \text{ClusterState.map.keys()} : \text{matching}(L, L_{pod})\}$

   For each matching labelset $L_m \in M$, let P_m be the set of all policies in L_m's selectPols and allowPols.

$$touchedLabelsets(L_{pod}) = M \cup \bigcup_{L_m \in M} \bigcup_{P \in P_m} touchedLabelsets(P)$$

So, we have seen that inconsistencies in the ClusterState (and Openstack resources) can occur, when handlers run concurrently that have an overlap in the labelsets they touch. We have also identified which labelsets are touched by each handler (as given by the *toucheLabelSets(pod)* and *toucheLabelSets(pol)* functions). This is an important step towards designing a solution.

4.0.5 Proposed Solution of the concurrency problem

In order to prevent the handlers that touch overlapping labelset from running together, we propose a per-labelset locking system, which would prevent this behaviour, allowing the system to only process events concurrently that do not lead to an inconsistent state.

Adapted Design

To realize this locking mechanism, we introduce a LockManager component to the system. The LockManager can be invoked by the Watchdog when handling new pods or policies. The adapted design can be viewed in Figure 4.1. Before handling any resource (pod or policy) the Watchdog now queries the involved labelsets and locks them in a thread-safe way. This is achieved by implementation with the Python thread module. This prevents any concurrent handling of the same ClusterState or Openstack resource. Note, that in this renewed design, the Watcher component has been replaced by the Kopf framework, since the framework handles all the event processing now.

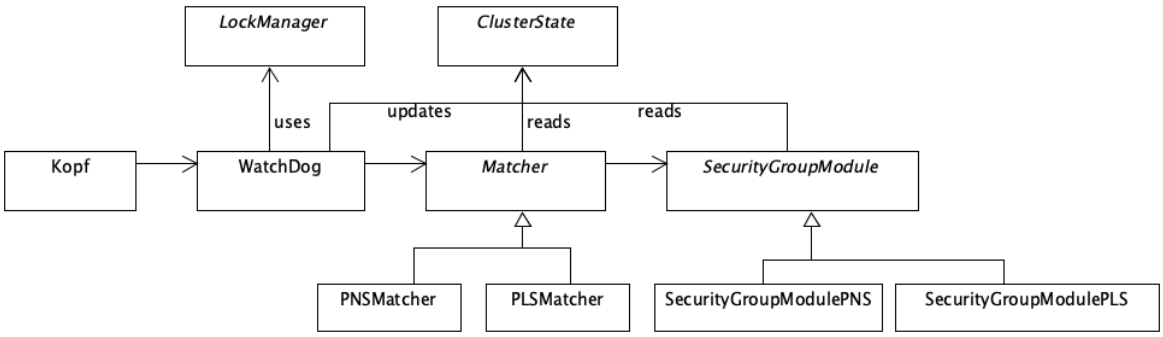

Figure 4.1: Grasshopper Adapted Design

Adapted Algorithm

In this subsection, we briefly go over the adapted HANDLENEWPOD(pod) and HANDLENEWPOLICY(pol) functions to illustrate how the locking system is integrated in the algorithm. Before handling any pod or policy resource in the Watchdog, we first query all the labelsets this handler will touch. Then, we lock all of these labelsets before proceeding to the handling of the event as before. The renewed algorithm handling a new pod (in the WatchDog), can be viewed in Algorithm 6

The adaptation in the HANDLENEWPOLICY(pol) is analogous.

Algorithm 6 Handle New Pod (Adapted with locking system)

```
1: function HandleNewPod(pod)
2:     Add pod to ClusterState.pods
3:     touchedLabelsets ← touchedLabelsets(pod)
4:
5:     Lock touchedLabelsets
6:
7:     for labelSet ∈ filter(λL: matching(L, pod), ClusterState.map.keys()) do
8:         mapEntry ← ClusterState.map[labelSet]
9:
10:        if pod.node ∉ mapEntry.matchNodes then
11:                                    ▷ Pod is the first pod on node to match labelSet
12:            Add pod.node to ClusterState.map[labelSet].matchNodes
13:            Call matcher.SGConfigNewPod(labelSet, pod.node)
14:        end if
15:    end for
16:
17:    Release touchedLabelsets
18:
19: end function
```

4.0.6 Implications on scalability

Even though the handling of pod and policy events where the handlers overlap in labelsets are prevented from running in parallel. There can still be a significant improvement in the efficiency of the program regarding the handling of non-overlapping resources. Consider for example the case where pods belonging to different application (and therefore having different labelsets) are auto-scaled at the same time. In this case the handlers for resources, belonging different applications can run in parallel. Therefore, in these cases, we expect a significant improvement in the scalability of the tool compared to the sequential processing that happens in the original version (V1).

4.0.7 Recovery and Start up

When the Kopf Operator first starts up, it starts by listing all events that happened in the cluster, by querying the K8s api-server. As mentioned in the introduction to Kopf, the framework writes the status of its handlers on the *.status* field, which adds the information of when an event of the resouce was handled (successfully or not). Kopf uses this information when it first starts up, to differentiate between events that happened before the Operator started and events that happen when the Operator was running. The *@on.resume* decorator can be used to handle resource that already existed in the cluster before startup. By using this decorator in addition to the *@on.create* and *@on.field(spec.nodeName)*, we can effectively process pods and policies that already existed in the cluster, allowing for the the program to start in an active cluster and to recover from failures. Additionally, the *@on.resume* handlers leverage Kopf's concurrency model as well, which is why we expect the start up and recovery process of this GH version (V2) to be faster.

4.0.8 Grasshopper Operator Pod

In the Operator approach a Pod gets deployed in the cluster. To achieve this, we've built a GH image that can run by a containerin a pod. This approach sovles the difficult setup process, since deploying the GH Operator is only one command away.

Eventhough it simplifies the setup process and allows for leveraging Kubernetes' pod management system, it also introduces an additional security risk. In order for GH to be able to use Openstack's API, it must be properly authenticated. This means that the application credentials must be present on the Pod (and therefore on the node). This opens op a new attack vector, since attackers can now potentially gain access to these credentials, in the case where he has gained access to the node where the Operator is running on.

5. Grasshopper Operator with Persistence

In this chapter, we describe the third version that we create of the GH program, in pursuit of improving its reliability features and scalability. In this version, we're specifically interested in improving its recovery time, which would in turn improve its overall reliability. The approach consists of persisting the central datastructure of the program (the ClusterState) to a database in the cluster. This approach would allow for faster recovery of the operator pod in case of a failure. This is because it can read in all information necessary for the program to work at recovery time, instead of having to reprocess all resources at startup time. This would decrease the amount of work the Operator has to do at recovery time. However, the drawback of this approach is the fact that all updates to the ClusterState's datastructures have to be written to permanent storage each time they change, hereby adding some overhead in the event processing. This could potentially result to larger latency times of the event handling.

5.0.1 ClusterState Persistence Implementation

This subsection explains how the persistence of the ClusterState was implemented. Figure 5.1 shows the updated design of the program. In this adapted design, the ClusterState component is replaced by the PeristedClusterState component, which has almost the exact same functionality of the ClusterState component. The component adds the functionality of writing its datastructures to a persistent storage. When an update happens to one of the ClusterState's datastructure, the component now additionally writes a serialised version of the whole datastructre to the storage.

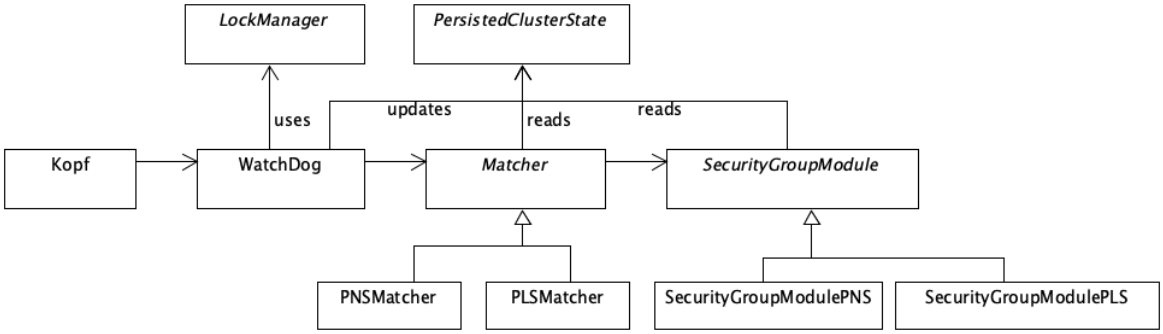

Figure 5.1: Grasshopper Adapted Design

This serialisation is achieved through Python's pickle module, which effectively serializes and deserializes Python objects into it's own bytestream format. [15] Method 1 en method 2 show how the implementation of the ClusterState and ClusterStatePersisted differ. It is clear that the new implementation additionally writes the datastructure to persistent storage. The change in functionality of the other methods adding or deleting elements from its datastructure is analogous.

Method 1 ClusterState.AddMapEntry(labelset, mapEntry) (ClusterState)

1: ClusterState.*map*[labelset] ← mapEntry

Method 2 ClusterStatePersisted.AddMapEntry(labelset, mapEntry) (PersistedClusterState)

1: ClusterState.*map*[labelset] ← mapEntry
2: **serialize** ClusterState.*map* **to** persistent storage

5.0.2 ClusterState's NFS share

The ClusterStatePersisted component persists its datastructures to permanent storage. This storage comes in the form on an NFS-based share, located on one of the nodes in the cluster. This share consists of .pkl files for each of the ClusterStatePersisted's datastructures, so that the component can write the serialized form to these files upon updates, illustrated in Method 2.

This share gets mounted into the Operator pod's filesytem by the Kubernetes system at boot-time. When updating one of the ClusterStatePersisted's datastructures, it simply writes the pickled object to the NFS-share, that is mounted in its filesystem.

5.0.3 Concerns & limitations

The current implementation serializes and writes the whole datastructure to storage each time and update happens. Additionally, it write the whole serialised datastructure to the storage. This could potentially cause a great amount of overhead in the case where the datastructures become very large, which may result in increased event handling latency.

These problems could potentially be improved by, for example, implementing a proper database system, allowing to write or remove single instances to the datastructure's storage or by implementing a separate daemon thread that periodically write the ClusterState to file. However, the purpose of the persistence for this context, was to research whether or not persisting the ClusterState is worth it in regards to improving the recovery time.

5.0.4 Expectations

Because of the fact that the Operator can read in the ClusterState, as opposed to having to reprocess resource events at startup time, we expect a significant improvement in recovery time. As explained in the previous subsection, the additional writing to storage may increase the event-handling latency of the program. In the next chapter, we intend to find out how this potential decrease in recovery time relates to the additional latency in event-handling.

6. Evaluation

In this chapter, we intend to evaluate the three different GH versions on their scalability and reliability properties and give an illustration of its effectiveness with a relevant example of a potential attack. We do this by designing experiments for each of the relevant domains, and running the experiment for each of the three versions. After the experiments are completed, we compare the test results achieved by the different versions and compare their performance in each of the domains. The goal of this evaluation, is to discover which versions perform best in each domain, so that we can learn which aspects of applying the Operator on GH are worthwile to integrate in its design.

In the sections that follow, we do a separate evaluation on each of the domains (scalability, reliability, security). We start by stating the aspect that will be measured and define the relevant research questions. Then, we describe the experiment setup and specify what exactly will be measured during the experiment. Lastly, we discuss the results obtained by the experiment and compare the performance of the different versions.

All of the experiments are carried out on KuLevuen's Openstack platform. The cloud platform, during the execution of the experiments, consisted of one master node and three worker nodes. The master node was VM composed of 2 virtual CPUs and 8GB of RAM, while the worker nodes consisted of 1 virtual CPU and 2GB of RAM.

6.0.1 Evaluating the scalability

In order to evaluate the scalability, we are interested in two aspects. The first one being: how does the system-usage (CPU utilisation and memory usage) on the master node behave during pod spikes of varying sizes. This will tell us something about the cost involved with running GH as an Operator in the cluster. We will measure GH's consumption on the cluster's resources, by measuring the usage of system resources on the master node. The second aspect we're interested ind is: how does the latency of pod events being handled scale accross various sized pod spikes. This is a fundamental aspect of the scalability of the GH program, because being able to respond quickly to pod events, will ensure the proper security group configuration, allowing these pods to communicate as allowed by the network policies. Not being able to respond fast enough to these events, will introduce additional application latency, since the pod's communication halts untill the right security group configuration is in place. Therefore, this is the most important aspect to research for the scalability of the program. The formal research questions are stated below.

RQ 1: *"How does consumption of the cluster's resources, scale accross different-sized pod creation spikes?"*

RQ 2: *"How does the latency of pod events being handled, scale accross different-sized pod creation spikes?"*

System usage experiment

At the beginning of the experiment one of the versions of GH is being started. Next, a measurer process starts, which is a Python script that periodically (e.g. every $1s$) measures the system's resource usage. Then, a cluster is being setup by applying network policies and pods in the cluster, during which the GH instance will be triggered to configure the security groups in the cloud platform accordingly. During this time, the measurer will measure the system's resources being used. Next, some time is being passed (40-60s), in order to give GH a sufficient amount of time to process all pod and policy events. Then, the measurer process gets terminated and the GH process or pod is being cleaned up.

Fair comparison

In order to give a fair comparison of the consumption of the cluster's resources by the different versions, we deploy the pods for V2 and V3 on the master node. This is because V1 runs as a Python module on the master node and when measuring its resources, it includes all components running on the master node, such as the K8s api-server. Therefore, we deploy the V2 and V3 pods on the master node as well, so that these measurements remain the same and the only difference in the the execution of the experiments, is the GH version.

Cluster setup

During the experiment, we simulate 3 applications running in the cluster: *app-a*, *app-b* and *app-c*. Each of these applications will have dedicated pods running during the experiment. Isolation between the different applications is realized by *DenyAll* policy that blocks communication between pods of different applications. The pods for each application, are classified as either a *frontend*, *backend* or *database* pod. Network policies will be applied to ensure isolation between them. Specifically, for each application, a network policy ensures that frontend pods can only communicate with backend pods and backend pods can only communicate with database pods. This results in 5 NPs per application. For the three applications in total, there will be 15 network policies applied during the execution of the experiments. After the NPs are applied, the test script deploys a burst of {10, 25, 50, 100} pods with labels that iterate through the applications and their roles within them. An illustration of the first 10 pods' labels is given below:

{'app': 'app-a', 'role': 'frontend'}, {'app': 'app-a', 'role': 'backend'}, {'app': 'app-a', 'role': 'database'}, {'app': 'app-b', 'role': 'frontend'}, {'app': 'app-b', 'role': 'backend'}, {'app': 'app-b', 'role': 'database'}, {'app': 'app-c', 'role': 'frontend'}, {'app': 'app-c', 'role': 'backend'}, {'app': 'app-c', 'role': 'database'}, {'app': 'app-a', 'role': 'frontend'}.

For the case where more than 10 pods are bursted, this cycle starts all over again.

In summary, we start up the GH version, start up the measurer process and start up the cluster simulation, which first sets up the 15 NPs and then bursts either 10, 25, 50 or 100 amount of pods. During this time the measurer process, measures system resources being used on the master node, as to measure GH's consumption of cluster resources. For each of the three versions and each burst-size, we perform 5 iterations of the experiment. The results obtained from these experiments are discussed below.

Results

For the test results of each iteration, the mean of the cpu utilisation and the memory usage accross the duration of the experiment is calculated. Then, for each burst size, the mean accross its iterations is calculated along with the standard deviation between iteration averages. The means across iterations for each burst size are plotted in the figures below, along with the standard deviations, which are expressed as the error handles. Figure 6.1 shows the results for the CPU utilisation, while Figure 6.2 shows the results for the memory usage.

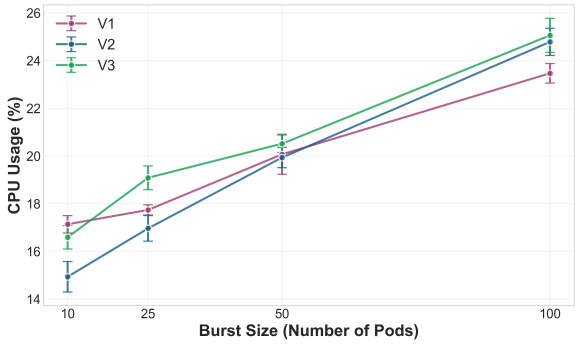

Figure 6.1: CPU Usage Comparison Across Different Burst Sizes

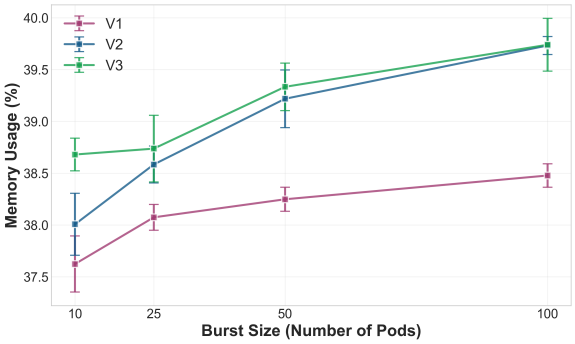

Figure 6.2: Memory Usage Comparison Across Different Burst Sizes

CPU utilisation performance comparison

The plot shows that V2 outperforms V1 on cpu usage for all burst sizes up to 50, after which it performs slightly worse. The performance of V2 and V3 converge to being more or less the same after burst size of 25. V3 has the highest utilisation accross all burst sizes, but is more outspoken in the smaller burst sizes. Overall, the cpu-utilisation appears largely the same across the different versions, since the largest difference in means between different version is around 2%.

Memory usage performance comparison

The memory usage plot shows that V2 and V3 consume slightly more memory than V1, which can be attributed to the fact that V2 and V3 process events in a multi-thread fashion, while V1 processes events sequentially. Running multiple threads concurrently consumes more memory than running only a single thread.

Event handle latency experiment

In this experiment we want to capture the pod event handle latency times, across various sized pod spikes, instead of measuring the systems resource usages. The experiment remains largely the same as the previous one. The main difference is the fact that the measurer process has been replaced by capturing pod creation- and handle-times. By combining these two times, the event-handle-latency time for each pod can be obtained.

Experiment setup

We begin by starting up the GH instance, then we setup the cluster's network policies and give GH some time to process them. Then the cluster simulator begins to burst pods in the same way as described in the previous experiment. The experiment's test script captures the times when pods have been created in the cluster, while the GH instance records the times when the pods have been handled. We combine these times in a .csv file joined on the pod name. Based on this information, we calculate the event handle latency times of the pods.

Event handle latency results

In the same way as before, we calculate the average event latency time for each iteration of each burst size. Next, we calculate the average latency times and standard deviations across iterations of a single burst size. This yields the plot shown in Figure 6.3, where the means are plotted along with the standard deviations, expressed as error bars, for various-sized pod creation spikes. For the visual results of this experiment, annotations showing the actual mean and standard deviation values are added for clarity.[1]

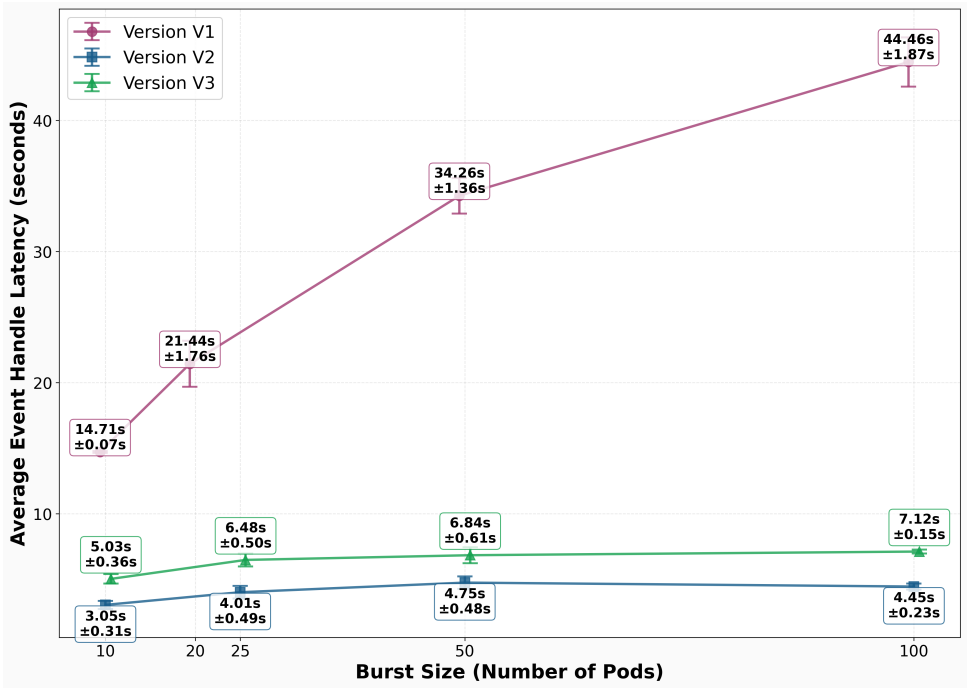

Figure 6.3: Event Handle Latency Comparison Across Different Burst Sizes

[1]This was not practical for the system usage plots

Event handle latency performance comparison

- It can be observed that V2 and V3 perform significantly better. This is an expected result, since they allow for concurrent processing of pods that not have overlapping labelsets. In the case of this experiment, pods belonging to different applications can be processed in parallel, hereby significantly improving handle latency times of pod events.

- V3 has an additional average latency of about 2-3s compared to V2. From our discussion in Section 5.0.3, additional latency is expected, due to the writing of the ClusterState to permanent storage. In the cluster setup of the experiment, the nfs-share is located on a worker node, while the GH pod is deployed on the master node. This may have introduced some network latency, which may have attributed to the additional 2-3s to the average pod handle event latency time.

6.0.2 Evaluating reliability

In this section we evaluate the three versions of GH for their reliability features. The most important one being the average recovery time of the program in case of failure. More formally, we're interested in the following research question:

RQ 3: *"How does the average recovery time of the three GH versions, scale with different sized clusters?"*

Experiment setup

We begin the experiment by starting an instance of GH (V1, V2 or V3). Then, we setup the cluster with network policies and pods in the same way is in the previous experiment. We simulate three applications runnning: *app-a*, *app-b* and *app-c*, with pods running for each of them as described before. The network policies, ensuring isolation between them and between the different pod roles within the applications are applied as well. In summary, we apply 15 NPs and setup the cluster with either 10, 25, 50 or 50 pods. During this time, GH processes the events and modifies security group configuration accordingly.

Then, we simuate a failure by killing the GH process (for V1) or deleting the GH pod (for V2 and V3). At this time, the security group configuration for all pods and policies are in place. When GH restarts, it has to regain knowledge of all resources in the cluster and events that happened to them. Lastly, we restart the GH instance and it reinitialize.

All three versions have different ways of reinitializing as described in their respective chapters. For clarity, we will briefly discuss them again here. V1 intializes the program by running the ClusterState's *initialize()* function, which requests all pod and policy resources in the cluster and processes them in the same way as V1 normally would. V2 utilizes the *@on.resume* decorators, which have the corresponding handlers attached to them, in order to process all resources that existed before the failure. V3 checks its database if GH was running or not. When it was, it calls the *loadClusterStateFromDb()* function. This function loads in ClusterState information stored in the database.

Results

We will now discuss the results that were obtained. The plot of the average recovery times in relation to the number of pods that were present in the cluster can be found in Figure 6.4.

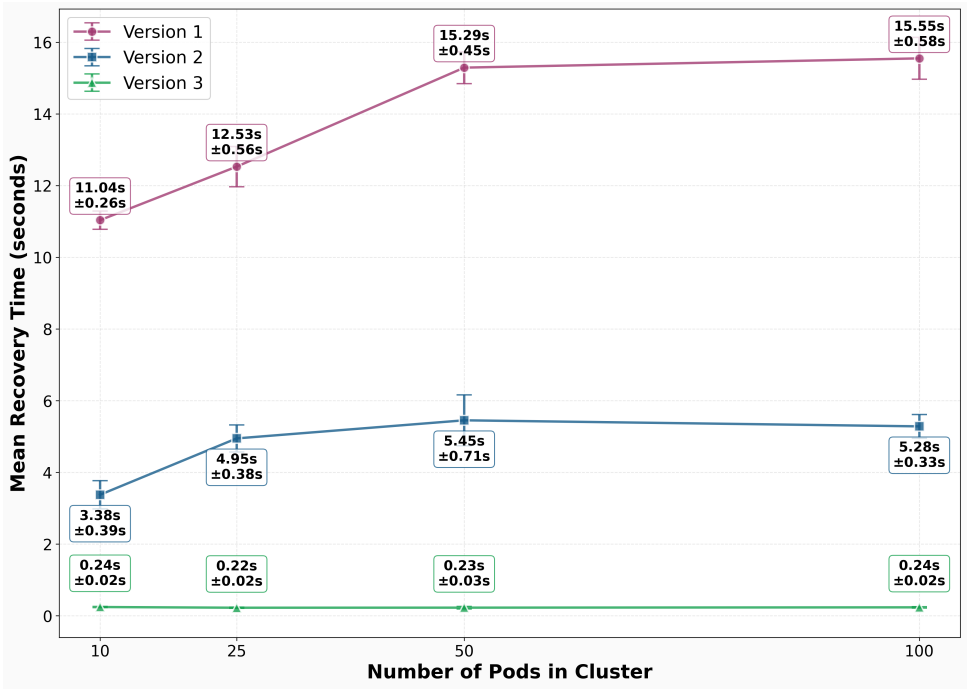

Figure 6.4: Average Recovery Times In Relation To Number of Pods

Recovery time performance comparison

- V1 performs the worst compared to the other versions. This can be attributed to the fact that it requests all pod and policy resources from the K8s api-server and proceeds to process them sequentially. Allthough, the performance is better than for the event handle latency, because security group configuration is already in place, and the program can skip costly Openstack API calls.
- V2 performs a lot better than V1 in the same way that V2 performed better than V1 in the event-handle-latency experiment. This is due to the fact that the *@on.resume* handlers are designed to run in parallel as well, hereby significantly speeding up the recovery process.
- V3 performs best, as contrary to the previous versions, it does not rely on K8s events at all in this case, as it can read in all cluster information from the persistent ClusterState storage. This significantly decreases the time necessary for the program to reinitialize after failure.

Important Note: In this experiment we did not consider pods or policies being created during GH's downtime. This allowed for a great recovery performance for V3, since it only has to read in the persisted ClusterState in this case. Should there be pods or policies created during downtime, the Operator would have to process these events as well after reading in the persistent storage. This could happen through the use of the *@on.resume* handlers in the same way as V2 does. This could increase the recovery time.

6.0.3 Security Illustration

In this subsection, we illustrate GH's effectiveness with a relevent example. In this scenario, a Kassandra Operator is running in the cluster in order to manage its pods. We illustrate this

security scenario with the Kassandra, because it is a popular choice for managing a distributed database system in a Kubernetes cluster. The illustration could, however, be given with any Operator having pod creation privileges in the cluster. This is usually the case, since the operator would be responsible for managing the application's specific pods.

Such an Operator introduces an additional security risk, in the case of compromised containers and elevation of its privileges, hereby escaping the container context. This is because the Operator is running with a Kubernetes ServiceAccount attached to it. This ServiceAccount has certain capabilities linked to it, such as for example the ability to create pods in a certain namespace. This ServiceAccount has a token that is stored in the Operator's pod's filesystem, which can be used by pods to proof to the kubernetes api-server that is has the account linked to them.

Possible lateral movement scenario

We assume an attacker has gained access to a node running a Kassandra Operator pod.

It now has access to this node's filesystem and can therefore access the pods' filesystems as well. It uses this access to extract the ServiceAccount tokens of the pods running on the node. It can test all of these tokens, to see if they have pod creation abilities. In the case where an attacker has compromised a container that is running on the same node the Operator pod is running on, the attacker can now gain access to the Operator's token, effectively giving him the ability to create pods in the cluster.

This ability can then used to create a reverse-shell attack. This is an attack, where the adversary listens on a given port on the node and creates a pod within the cluster, that has the only purpose of creating a shell connection to the node. When the pod is deployed, the attacker now has a shell connection to a pod on another node in the cluster. The attacker could now escape the container context (e.g. by mounting to the nodes /root filesystem) and gain access to the node. The attacker has now gained access to another node in the cluster, effectively performing a lateral movement attack.

Static Security Group Scenario (No GH)

In this case, we assume a recursive static application security group is in place, attached to all nodes in the cluster, allowing application traffic to and from every node. This enables the Kubernetes system to schedule application pods on every node and these pods to be able to communicate with all other application pods. However, there is no enforcement of least-privilege in place, in the sense that nodes that do not host such applications, will still be allowed communication on the application port.

Dynamic Security Group Scenario (GH)

In this scenario, an instance of GH is running, dynamically configuring security groups in order to allow all node communication that is required by the application. This required communication is expressed through appriopriate network policies in the cluster. When application pods get scheduled in the cluster, GH ensures the proper security group configuration is in place in order to allow application communication on the nodes as well.

Cluster Setup

In both scenarios the cluster is setup with an application running on two worker nodes in the cluster. This application consists two pods: *application-pod-1* and *application-pod-2* each running on a different node, where *application-pod-1* runs on the worker and *application-pod-2* runs on the worker-1 node. They both have the labels *'app: 'example-app'* assigned to them

and there is a network policy in place, allowing ingress and egress traffic on port *8080* (the application port).

Experiment Setup

In both scenarios we setup the application in the cluster, by applying the application pods and the network policy, allowing application traffic between application pods. In the static case, we setup the security group configuration as described above. In the GH case, we run an instance of GH.

Then we try to perform a reverse-shell attack on the nodes in the cluster. We assume an attacker has gained access to the worker node, on which the Kassandra Operator pod is running on as well. Then we execute the rev-shell-attack.sh script, which will perfrom a reverse shell attack on a given node in the cluster. It first checks for tokens in the pods' filesystem on the node and extract the ones that have pod-creation abilities. It finds the token from the Kassandra Operator pod. It then uses this token, to make a request to the K8s api-server to create a reverse-shell pod, which only purpose is to create an ncat connection on port *8080*. It then opens the *8080* port on the node from which the attack originates. When the ports are opened on the nodes, the connection eventually succeeds.

We run the rev-shell-attack script in both scenarios to the worker-1 and worker-2 nodes. Note, that the attack originates from the worker node.

Results

- **Static Case (No GH):** As expected, the attack succeeds for both the worker-1 and worker-2 node in this case. This is because the application port (*8080*) is by default opened on every node in the cluster.
- **Dynamic Case (GH):** The attack succeeds for the worker-1 node, since there is an application pod running on it, which is allowed ingress and egress traffic on port (*8080*) by the network policy. The GH instance has taken notice of the policy and the pods' schedulings and has created an SG that allows ingress and egress traffic on *8080* as well and has attached it to the worker and worker-1 node, which allows the attack to succeed for the worker-1 node.

  However, the attacked is blocked for the worker-2 node, since there was not any application pod scheduled on there. Therefore, GH has not attached the application SG to this node, effectively blocking the lateral movement.

This section showed a relevant lateral movement attack scenario and the experiment that carried out this scenario effectively showed GH blocking the attack for any node that did not have an application pod running on it. The experiment successfully showed that GH enforces least-privilege on the cloud layer, as defined by the network policies and shows that this can reduce lateral movement attacks in a cloud environment.

6.1 Discussion

We have illustrated GH's effectiveness in reducing lateral movement attacks by reviewing a relevant attack scenario in the section above. The question that remains is the following: Is the program scalable to large clusters and potentially large pod creation spikes? When looking at the event-handle-latency experiment, we can see that V1 is lacking in its ability to handle such

large pod spikes, with the average pod creation handle times of 34.26s for a burst of 50 pods. This could cause additional application latency, since pods may be running on a nodes that not yet have appriopriate security group configuration to allow required communication.

We saw a signifant increase in its scalability performance with V2, reaching an average pod creation handle-time of 6.84s, which proofs to be a much more promising result. Therefore, it seems that applying the Operator pattern to the GH program and more specifically, leveraging Kopf's concurrency model can lead to a significant increase in its scalability.

The reliability experiment showed that writing the ClusterState to a permanent storage, may significantly improve GH's recovery time, which is an important reliability feature to have in real-world scenarios. Alltough we did not consider pods and policies being created during Operator downtime, the results obtained from this experiment may still be valuable in showing the effectiveness of writing the ClusterState to permanent storage, hereby decreasing its recovery time. This is because we assume the number of resources being created at Operator up-time to be greater than the number of resources created at Operator downtime. This would result in the amount of processing time needed by the *@on.resume* handlers to still be limited compared to the V2 version.

We saw however, that writing the ClusterState to persistent storage comes with an event-handle latency cost in our current implementation.

This is because each time the ClusterState gets updated, that whole ClusterState's datastructure gets written to database in our current implementation of V3, which is not very efficient nor scalable.

However, this could be improved by e.g. implementing a separate daemon thread that per that periodically writes the ClusterState to database. Should such an alternative approach be successful, it might still be worthwile to write the ClusterState to permanent storage, considering the decrease in recovery time that can be achieved by using this approach.

7. Conclusion

In chapter 1 we introduced the contex of our problem domain. We have seen that the cloud native paradigm is a highly popular approach for deploying applications in a fast, scalable and cost-effective way. This is because of the multi-tentant nature of cloud platform and their leveraging of the fast spin-up times of containers. This setting, however introduces some security challenges of which we have seen a specific example, which motivates the need of enforcing network policies upon the cloud layer in a least-privilege way.

We have introduced GH and reviewed its algorithm and design for configuring security groups in this setting, based on the network policies that are in place and pod scheduling decisions. We illustrated its effectiveness in reducing lateral movement attacks in section 6.0.3, where we reviewed a concrete post-exploitation scenario involving a reverse shell attack on the nodes in the cluster. We illustrated GH effectively blocking this attack for nodes that did not have any pods running on them, which would result in GH opening up ports used in the attack.

In chapter 4, we introduced a new version (V2) of GH which applies a part of the Operator pattern and leverages a specific Operator framework's (Kopf) concurrency model, which was used to improve its scalability features. We reviewed the adaptations necessary in order to enable the GH algorithm to allow for concurrent processing. Based on this examination, we implemented a per-labelset locking system, which locks all labelsets that will be accessed or modified in the ClusterState's *map* during the handling of the event. This prevents harmful concurrent processing of events, that access or modify shared resources. In our evalutation in chapter 6 and more specifically in section 6.0.1, we have seen that enabling concurrent processing of event, significantly improved the scalability of the program, in the case where pods of different applications where bursted.

In chapter 5 we extended the second version that resulted in the third version (V3), which leverages persistence of the ClusterState to permanent storage, in order to reduce its recovery time, hereby improving its reliability. We showed that doing this, can significantly decrease recovery time of the program. However, we also showed that in our current implementation, this came at the cost of additional event-handle latency, in which case it might be questionable if it is worthwhile to do. We also mentioned though, that this implementation can potentially be improved by taking a more efficient approach (such as e.g. the separate daemon), in which case there could be a real gain in recovery time, without the additional latency cost.

In the first experiment we showed that the V2 and V3 versions (in our experiments) did not show any significant increase of consumption of the cluster's resources. We can therefore conclude that the V2 and V3 implementations did not introduce any significant additional cost.

In summary, applying aspects of the Operator pattern to the GH program, seems to be a worthwhile approach for creating a more scalable and reliable GH version.

Bibliography

[1] Amazon Web Services. *What is Cloud Native?* URL: https://aws.amazon.com/what-is/cloud-native/.

[2] Red Hat. *What is container orchestration?* Mar. 21, 2025. URL: https://www.redhat.com/en/topics/containers/what-is-container-orchestration.

[3] The Kubernetes Authors. *Kubernetes: Production-Grade Container Orchestration*. URL: https://kubernetes.io/.

[4] Brendan Burns et al. "Borg, Omega, and Kubernetes". In: *ACM Queue* 14 (2016), pp. 70–93. URL: https://research.google.com/pubs/archive/44843.pdf.

[5] Katrine Wist, Malene Helsem, and Danilo Gligoroski. "Vulnerability Analysis of 2500 Docker Hub Images". In: *Advances in Security, Networks, and Internet of Things*. Springer, 2021. DOI: 10.1007/978-3-030-71017-0_22.

[6] Rui Shu, Xiaohui Gu, and William Enck. "A Study of Security Vulnerabilities on Docker Hub". In: *Proceedings of the 7th ACM Conference on Data and Application Security and Privacy (CODASPY)*. ACM, 2017, pp. 269–280. DOI: 10.1145/3029806.3029832. URL: https://enck.org/pubs/shu-codaspy17.pdf.

[7] OpenStack Foundation. *OpenStack: The Most Widely Deployed Open Source Cloud Software in the World*. URL: https://www.openstack.org/.

[8] The Kubernetes Authors. *Kubernetes Components*. URL: https://kubernetes.io/docs/concepts/overview/components/.

[9] CNCF TAG App Delivery Operator Working Group. *CNCF Operator White Paper, Version1.0*. Version1.0. Cloud Native Computing Foundation. July 2021. URL: https://github.com/cncf/tag-app-delivery/blob/main/operator-wg/whitepaper/Operator-WhitePaper_v1-0.md.

[10] Sergey Vasilyev. *Kopf: Kubernetes Operator Pythonic Framework*. URL: https://kopf.readthedocs.io/en/stable/.

[11] Alaa Oqaily et al. "NFVGuard: Verifying the Security of Multilevel Network Functions Virtualization (NFV) Stack". In: (). URL: https://arc.encs.concordia.ca/papers/NFVGuard.pdf.

[12] Yifan Li et al. "Kano: Efficient Cloud Native Network Policy Verification". In: (). URL: https://ieeexplore.ieee.org/document/9990602.

[13] Jaehyun Nam, Seungsoo Lee, and Hyunmin Seo. "BASTION: A Security Enforcement Network Stack for Container Networks". In: (). URL: https://www.usenix.org/system/files/atc20-nam.pdf.

[14] Kopf Contributors. *Kopf Framework - Event Queueing and Worker Implementation*. Source code implementing resource-based concurrency model. URL: https://github.com/nolar/kopf/blob/main/kopf/_core/reactor/queueing.py.

[15] Python Software Foundation. *pickle — Python object serialization*. URL: https://docs.python.org/3/library/pickle.html.

DEPARTMENT OF COMPUTER SCIENCE
Celestijnenlaan 200A bus 2402
3001 LEUVEN, BELGIË
tel. + 32 16 32 70 10
fax + 32 16 32 79 96
www.kuleuven.be